

Sesión 10
Transformación de variables y matriz analítica

Docente: Soledad Espezua Llerena

s.espezual@up.edu.pe

Objetivos de la sesión

- Detectar y analizar valores atípicos, crear variables derivadas con sentido para el problema, aplicar discretización y escalamiento cuando sean necesarios, y construir una matriz analítica final adecuada para el análisis posterior.
- Aprender a tomar decisiones justificadas según el significado de las variables, la unidad de análisis y el objetivo del proyecto, evitando transformar los datos de manera automática.

Proceso hacia una matriz analítica

Base original

Pacientes · Atenciones · Medicación

Base limpia e integrada

Una fila por unidad de análisis

- ✓ Diagnóstico de calidad
- ✓ Limpieza estructural
- ✓ Integración de tablas
- ✓ Unidad de análisis
- ✓ Exploración y visualización

Transformación

Codificar · derivar · discretizar

- ✓ Decidir qué hacer con faltantes
- ✓ Eliminar o imputar cuando corresponda
- ✓ Codificar variables categóricas
- ✓ Construir una primera representación de matriz analítica

Matriz analítica

Variables listas para aplicar técnicas de minería

- Detectar y decidir qué hacer con valores atípicos
- Crear características con sentido analítico
- Discretizar cuando aporte interpretabilidad
- Escalar variables numéricas
- Construir la matriz analítica final.

Docente: Soledad Espezúa Llerena

Matriz analítica integrada

Es una tabla organizada para el análisis posterior, donde cada fila representa una unidad de análisis y algunas variables pueden haber sido transformadas para obtener una representación adecuada.

Ejemplo: Matriz analítica a nivel paciente

Conservar para trazabilidad

Atenciones agregadas a paciente

Detalle- agregado a paciente

	id_paciente	edad	sexo_bin	asegurado_bin	espera_promedio_imp	n_atenciones	tuvo_reingreso	n_prescripciones	n_medicamentos_distintos	distrito_imp
0	P001	64.0	0	1	32.5	2	0	3	3	Surco
1	P002	45.0	1	0	15.0	1	1	2	2	San Borja
2	P003	70.0	0	1	32.5	2	0	3	2	Miraflores
3	P004	34.0	0	1	22.0	2	0	2	2	Lince
4	P005	28.0	1	0	12.0	1	0	2	2	Surquillo
5	P006	52.0	0	1	22.0	1	0	2	2	Desconocido
6	P007	39.0	1	1	28.0	2	0	3	3	San Miguel
7	P008	61.0	0	0	28.0	1	1	1	1	Pueblo Libre
8	P009	50.0	1	1	22.0	2	0	3	3	Jesús María
9	P010	33.0	0	1	22.0	1	0	1	1	Barranco
10	P011	72.0	1	1	41.5	2	1	5	4	La Molina
11	P012	26.0	0	0	20.0	1	0	1	1	San Borja

Detección y tratamiento de valores atípicos

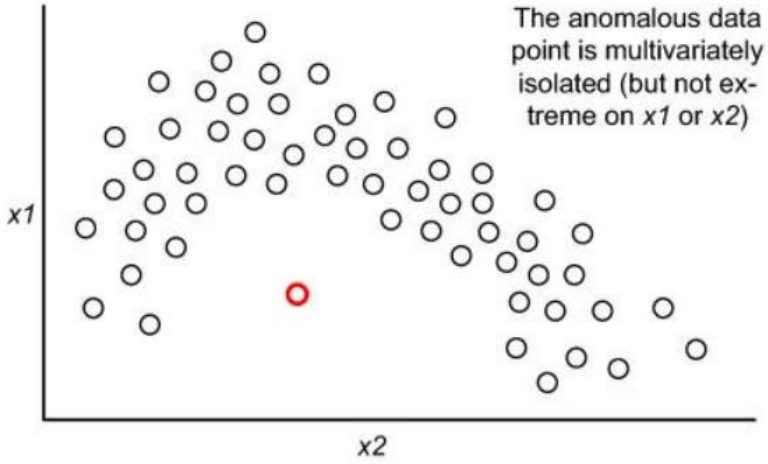


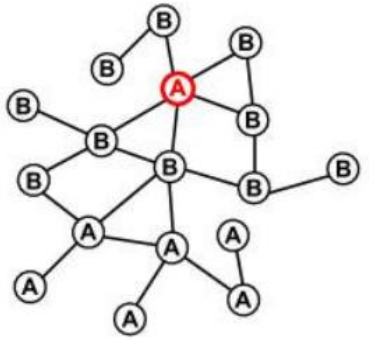
¿Qué son Outliers?

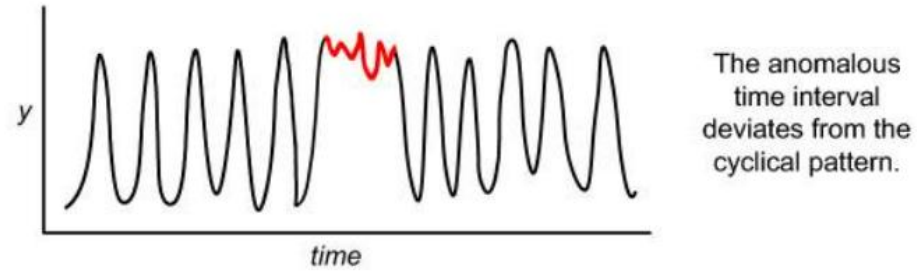
Son ocurrencias de datos que de alguna manera son inusuales, raros, distintos y no se ajustan o no encajan con los patrones generales.

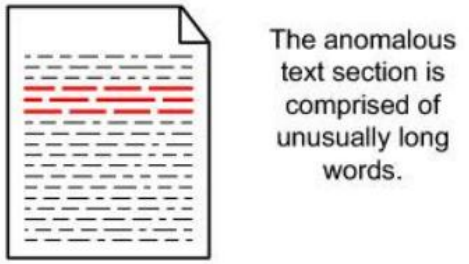
Definiremos valores outliers como valores atípicos, que se “escapan al rango en donde se concentran la mayoría de datos”.

Algunas otras formas de denominar un “outlier” son : valores **Anormales** o **Anómalos**.









Fuente: Foorthuis R. (2021). On the nature and types of anomalies: a review of deviations in data. International journal of data science and analytics, 1–35. Advance online publication. https://doi.org/10.1007/s41060-021-00265-1

Valor atípico no significa necesariamente error

El tratamiento de valores atípicos depende del contexto, no solo de una regla estadística

- Puede deberse a un error de registro

  Ejemplo. Al ingreso anual de una persona se le agrega accidentalmente un cero y el valor registrado pasa a ser 10 veces mayor.

- Puede originarse por una falla del instrumento de medición

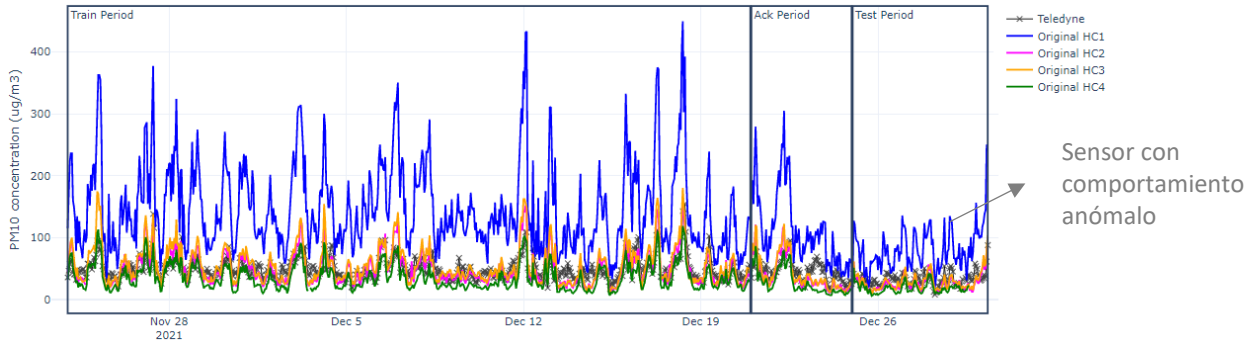
  Ejemplo. De cuatro sensores que miden contaminación, uno presenta falla y genera valores anómalos (línea azul)

- Puede ser un caso real pero poco frecuente

  Ejemplo. Registros deportivos de deportistas excepcionales pueden aparecer como atípicos respecto al resto de los registros.

- Puede revelar un subgrupo relevante o una situación excepcional.

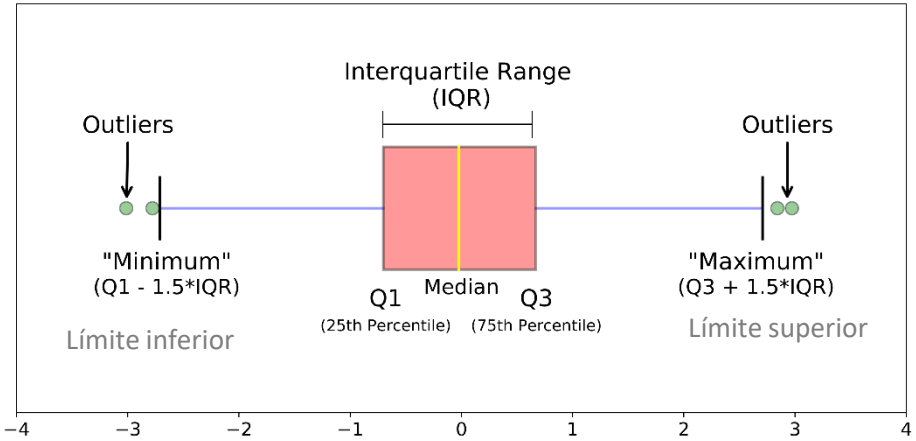
  Ejemplo: En un conjunto de clientes, compras muy altas podrían corresponder a clientes mayoristas.



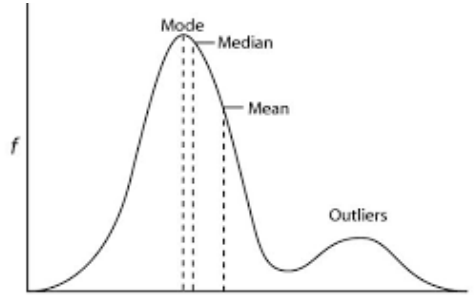
Fuente imagen sensor. Villanueva, E.; Espezua, S.; Castelar, G.; Diaz, K.; Ingaroca, E. Smart Multi-Sensor Calibration of Low-Cost Particulate Matter Monitors. Sensors 2023, 23, 3776. https://doi.org/10.3390/s23073776

Formas de detectar valores atípicos

Visualización → Permite identificar asimetrías, concentraciones y observaciones extremas.



Los valores extremos suelen aparecer alejados de la mayor concentración de observaciones



Regla estadística → Define un criterio reproducible

Regla IQR: valores < (Q1 - 1.5 x IQR) o valores > (IQ3 + 1.5 x IQR) se consideran candidatos a valores atípicos.

Reglas de negocio → Detecta valores imposibles o inverosímiles

Peso (kg)	Edad
250	15
60	44
70	5
55	21

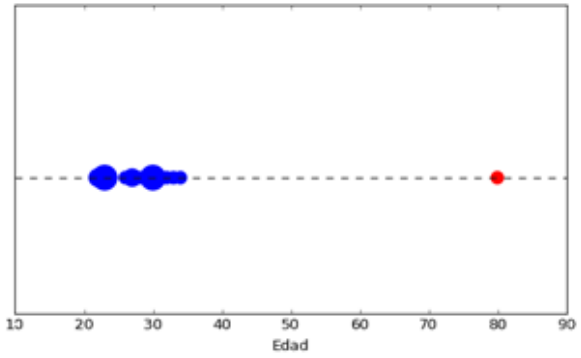
Detectar un valor atípico no determina qué hacer con él.
Primero lo identificamos; luego investigamos su significado.

Un valor atípico depende también de la combinación de variables

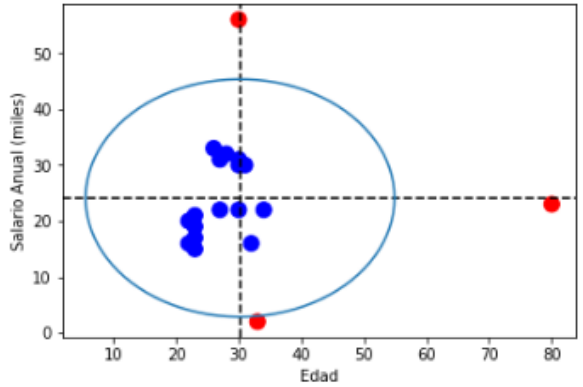
Un registro puede ser atípico en una variable o por la combinación de varias variables.

	Edad	Salario	Compras
0	22	16	1
1	22	20	2
2	23	15	1
3	23	21	20
4	23	19	1
5	23	17	0
:	:	:	:
17	34	22	2
18	80	23	2

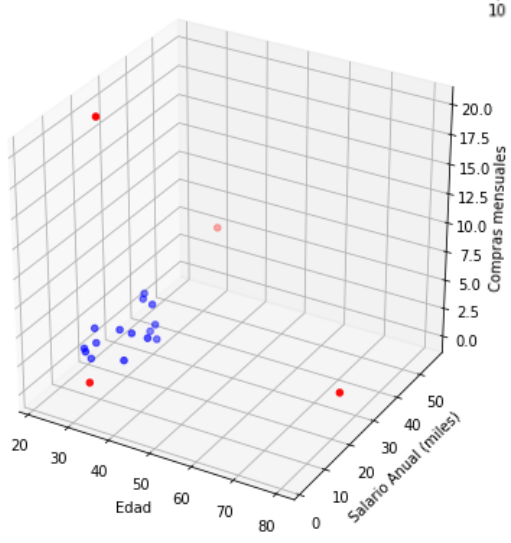
Edad: Análisis univariado



Edad vs Salario anual: Análisis multivariado



Edad vs Salario anual vs Compras mensuales



Rojo: observaciones potencialmente atípicas

Fuente: https://www.aprendemachinelearning.com/deteccion-de-outliers-en-python-anomalia/

Regla del rango intercuartílico (IQR)

Una regla simple y robusta para detectar posibles outliers univariados.

IQR = Q3 – Q1

```
q1 = atenciones["espera_min"].quantile(0.25)
q3 = atenciones["espera_min"].quantile(0.75)

iqr = q3 - q1
limite_inferior = q1 - 1.5 * iqr
limite_superior = q3 + 1.5 * iqr

outliers_espera = atenciones[
    (atenciones["espera_min"] < limite_inferior) |
    (atenciones["espera_min"] > limite_superior)
]

display(outliers_espera)
```

Cualquier valor, que esté más allá del rango IQR e considera candidato a valor atípico.

```
def loc_outliers(dfx, columnas_lst):
  for columna in columnas_lst:
    q1 = df[columna].quantile(0.25)
    q3 = df[columna].quantile(0.75)
    iqr = q3 - q1
    ul = q3 + 1.5*iqr # upper_limit
    ll = q1 - 1.5*iqr # lower_limit
    dfx.loc[dfx[columna]>ul, columna] = ul
    dfx.loc[dfx[columna]<ll, columna] = ll
  return dfx
```

IQR funciona bien cuando la distribución es asimétrica o tiene valores extremos, porque usa cuartiles.

Z-score

Mide cuántas desviaciones estándar se encuentra un valor respecto de la media.
Por ejemplo, con un umbral de 3, serían candidatos los valores con $| z |> 3$.

$$z = \frac{\mathrm{x} - mean(x)}{stdev(x)}$$

El umbral de cuantas desviaciones estándar, se define según el contexto;
son comunes valores como 2, 2.5 o 3,

```
media = atenciones["espera_min"].mean()
desv = atenciones["espera_min"].std()

atenciones["z_score"] = (atenciones["espera_min"] - media)/desv


outliers_z = atenciones[
                    atenciones["z_score"].abs() > 3
            ]

display(outliers_z)
```

Z-score depende de la media y la desviación estándar, por lo que es más sensible a valores extremos y tiene más sentido cuando la distribución es aproximadamente simétrica.

Detectar no significa eliminar

No debemos eliminar un registro solo porque una regla estadística lo marcó como outlier.

.

Situación	Caso	¿Es necesariamente un error?	Decisión razonable
Error evidente	Edad = 250 años	Probablemente sí	Corregir si existe una fuente confiable; si no, considerar el valor como faltante.
Valor extremo pero válido	Compra 20 veces mayor al promedio	Depende del contexto	Conservar, transformar o limitar su influencia, según el objetivo del análisis.
Valor dudoso	Dosis de medicación inusual	Depende del contexto	Investigar antes de modificar
Valor extremo válido que afecta mucho la escala	Espera = 180 min en emergencia	No necesariamente	Conservar si se confirma que es un valor válido.

Puede ser error o un caso real de saturación; la decisión no puede tomarse solo por ser estadísticamente atípico.

Creación de variables derivadas

A	B	A + B
1	2	3
3	4	7
5	6	11
7	8	15

Tipos frecuentes de variables derivadas

Una variable derivada se obtiene a partir de una o más variables existentes para representar un aspecto útil del problema.
Crear una variable derivada permite construir una nueva característica analítica a partir de información existente.

Tipo	Idea	Ejemplo en el caso clínico
Indicador	Resume presencia / ausencia de un evento	tuvo_reingreso, recibio_antibiotico
Temporal	Se obtiene a partir de fechas o del orden temporal de los registros.	tipo_ultima_atencion, días desde última atención
Agregada	Resume varios registros para construir una característica de la unidad de análisis.	n_atenciones, espera_promedio
Combinada	Relaciona dos o más variables para representar una nueva característica	tasa = eventos / periodo

Tipos frecuentes de variables derivadas

Ejemplo 1: ¿el paciente tuvo al menos un reingreso?

```
# Convertimos el indicador de cada atención a 0/1.
atenciones["reingreso_bin"] = (
    atenciones["reingreso"]
    .map({"No": 0, "Sí": 1})
)

# Si alguna atención tiene reingreso = 1, el máximo será 1.
reingreso_paciente = (
    atenciones
    .groupby("id_paciente")
    .agg(tuvo_reingreso=("reingreso_bin", "max"))
    .reset_index()
)
```

No es solo una recodificación: estamos creando una nueva característica del paciente.

Interpretación

- La variable original está en la atención.
- La nueva variable está en el paciente.
- max() funciona como “alguna vez ocurrió”.

Tipos frecuentes de variables derivadas

Ejemplo 2: tipo de la última atención

Una característica temporal puede construirse seleccionando el registro más reciente.

```
ultima_atencion = (
    atenciones
    .sort_values("fecha")
    .drop_duplicates("id_paciente", keep="last")
    [["id_paciente", "tipo_atencion"]]
    .rename(columns={
        "tipo_atencion": "tipo_ultima_atencion"
    })
)
```

Luego, como es nominal, tipo_ultima_atencion podría codificarse con one-hot encoding.

Antes		Después
Varias atenciones por paciente	→	Una fila por paciente
tipo_atencion	→	tipo_ultima_atencion
La fecha define el orden	→	Se conserva el tipo del registro más reciente

Discretización

Discretizar significa transformar una variable numérica en categorías definidas por intervalos.

Discretización por cuantiles: pd.cut()

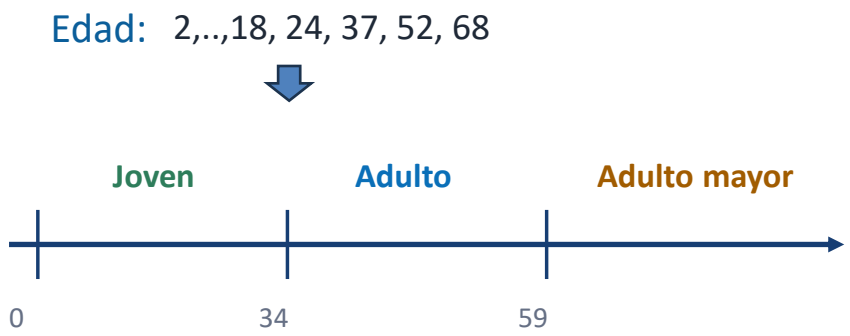
cut(): sirve para discretizar usando intervalos definidos por nosotros.

```
base_paciente["grupo_edad"] = pd.cut(
    base_paciente["edad"],
    bins=[0, 29, 59, float("inf")],
    labels=["Joven", "Adulto", "Adulto mayor"]
)

display(
    base_paciente[["edad", "grupo_edad"]]
)
```

grupo_edad es una variable derivada ordinal.

Ventaja	Riesgo
Los cortes son fáciles de interpretar	Algunos intervalos pueden quedar con pocos casos
Puede usar conocimiento del dominio	Los resultados dependen de los puntos de corte



Discretización por intervalos definidos: pd.qcut()

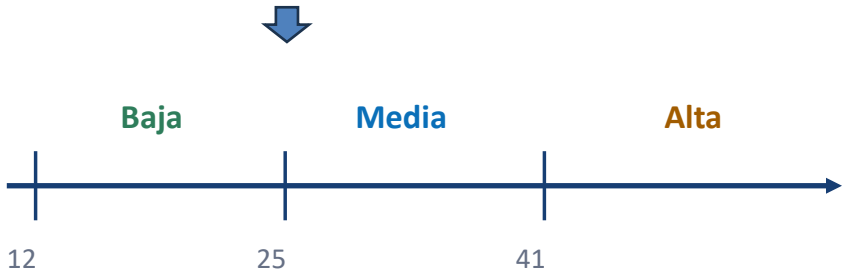
qcut(): sirve para discretizar según cuantiles, buscando grupos con cantidades parecidas de observaciones.

```
base_paciente["espera_grupo"] = pd.qcut(
    base_paciente["espera_promedio"],
    q=3,
    labels=["Baja", "Media", "Alta"]
)

display(
    base_paciente[["espera_promedio", "espera_grupo"]]
)
```

Ventaja	Riesgo
Los grupos tienden a tener tamaños similares	Los límites dependen de los datos
Útil para comparar segmentos relativos	El mismo corte puede cambiar en otra muestra

espera_promedio: 12, …. 41 min



qcut() define los grupos por posición en la distribución, no por significado clínico.

Escalamiento

El escalamiento es especialmente importante antes de técnicas que comparan observaciones mediante distancias o que dependen de la variabilidad de las variables, como K-means, DBSCAN o PCA.

Escalar

Variables con rangos distintos pueden aportar magnitudes muy diferentes al análisis.

Variable	Rango aproximado
n_atenciones	1 – 6
edad	18 – 85
espera_promedio	10 – 60
ingreso anual (otro caso)	10 000 – 200 000

Escalar no “mejora” los datos: los pone en una escala comparable.

- El escalamiento no cambia el orden de los casos.
- Sí cambia la magnitud numérica utilizada por técnicas posteriores.
- Es especialmente relevante cuando se comparan observaciones mediante distancias o magnitudes.

Normalización min–max

Min–Max reescala una variable, usualmente al rango [0,1].
Mantiene el orden relativo de los valores, pero puede verse afectado por outliers.

$$z = \frac{\mathrm{x} - min(x)}{[\max(x) - min(x)]}$$

	Salario	Edad	Publicaciones
0	2000	23	3
1	5000	30	15
2	10000	41	4
3	15000	35	5

Min-Max

	Salario	Edad	Publicaciones
0	0.000000	0.000000	0.000000
1	0.230769	0.388889	1.000000
2	0.615385	1.000000	0.083333
3	1.000000	0.666667	0.166667

- Aplicable si los datos NO siguen una distribución normal.
- Es útil cuando se desea comparar variables que tienen diferentes escalas o unidades de medida, o cuando se desea mejorar el rendimiento de algunos modelos de ML

```
from sklearn.preprocessing import MinMaxScaler

variables = [
    "edad",
    "espera_promedio",
    "n_atenciones"
]

scaler = MinMaxScaler()

base_minmax = base_paciente.copy()
base_minmax[variables] = scaler.fit_transform(
    base_minmax[variables]
)
```

Estandarización z-score

Z-score: altera los valores de cada columna para que tengan media = 0 y desviación estándar = 1

$$z = \frac{\mathrm{x} - mean(x)}{stdev(x)}$$

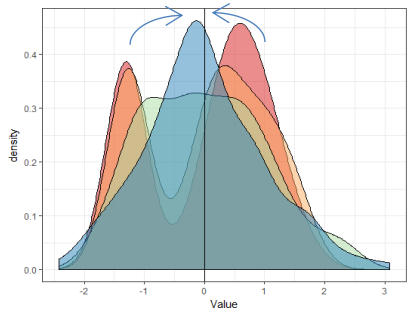
Expresa cada valor en unidades de desviación estándar respecto de la media.

	Bacteria 1	Bacteria 2	Bacteria 3
0	-220	300	123
1	-4500	-200	115
2	30	-450	234
3	-300	500	905

Zscore

	Bacteria 1	Bacteria 2	Bacteria 3
0	0.546028	0.691223	-0.676300
1	-1.728423	-0.625393	-0.700754
2	0.678881	-1.283701	-0.337004
3	0.503515	1.217870	1.714057

- Es útil para comparar los valores de la variable en diferentes momentos.




```
from sklearn.preprocessing import StandardScaler

variables = [
    "edad",
    "espera_promedio",
    "n_atenciones"
]

scaler = StandardScaler()

base_z = base_paciente.copy()
base_z[variables] = scaler.fit_transform(
    base_z[variables]
)
```

¿Cuándo usar Min-Max o Z-score?

Quiero todas las variables entre 0 y 1

Quiero comparar valores respecto de su media

	Min–Max	Z-score
Resultado	Rango usual [0,1]	Media 0, desviación estándar 1
Interpretación	0 = mínimo, 1 = máximo	Distancia respecto de la media en desviaciones estándar
Outliers	Muy sensible	También sensible, porque usa media y desviación estándar
Cuándo usar	Cuando interesa un rango definido	Cuando interesa centrar y comparar magnitudes

De la base integrada a la matriz analítica

Base integrada

```
id_paciente
sexo
edad
asegurado
n_atenciones
espera_promedio
fecha_primera_atención
fecha_ultima_atención
n_medicamentos_distintos
```

Unidad de análisis: paciente

codificar
derivar
discretizar
escalar



Matriz analítica

```
id_paciente (trazabilidad)
sexo_bin
edad
asegurado_bin
grupo_edad_ord
ult_atencion_Consulta
ult_atencion_Control
ult_atencion_Emergencia
n_atenciones
espera_promedio_imp
dias_seguimiento
n_medicamentos_distintos
recibio_antibiotico
```

¿Conservo o elimino las variables originales?

En un proyecto real se debe evaluar si conviene conservar simultáneamente la variable original y su versión discretizada.

Mantener	Evaluar / transformar	Excluir de las características
Variables relevantes	Categóricas → codificación	Identificadores
Variables derivadas justificadas	Numéricas → escalamiento si corresponde	Variables sin utilidad analítica
Variable original o discretizada según el objetivo	Outliers / faltantes según decisión	Variables redundantes sin justificación

Buena práctica: conservar la base interpretativa y construir una copia transformada para el análisis.

UNIVERSIDAD DEL PACÍFICO

¡Gracias por su atención!